Time complexity

It tells us how long an algorithm takes to run as the input size increases.

It describes the growth rate of an algorithms times not the actual time in seconds.

Ways to find TC → Experimental Analysis
→ Assymptotic Notation

Experimental Analysis

Assymptotic Notation

Depends on the user input, not on system.

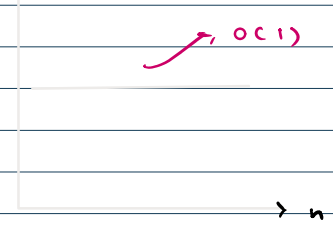
No. of times a repetitive statement is running.

AN → Worst (Big-O) O
→ Best (omega) Ω
→ Average (Theta) Θ

Time complexity → O(1)

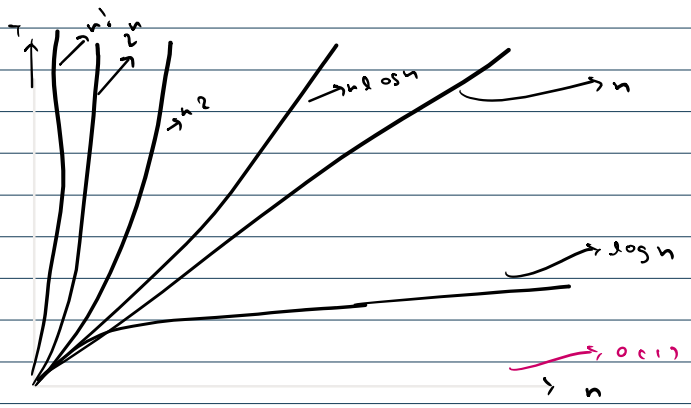
mathematical expressions
logical operators
bitwise operators
arithmetic operator
relational operators
variable declaration
variable assignment

T → Time
n → Input size



$f(n) = 5n^2 + 2n^5 + 3n + 2$

$f(n) = 2^n + 2n^5 + 3n + 2$




```
System.out.println("Hello Akarsh");
System.out.println("Hello Akarsh");
System.out.println("Hello Akarsh");
System.out.println("Hello Akarsh");
System.out.println("Hello Akarsh");
System.out.println("Hello Akarsh");
```

Best | worst => O(5) = O(1)

```
// Linear Search
public static int linearSearch(int[] arr, int item) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == item) {
            return i;
        }
    }
    return -1;
}
```

n + 1 + 1 => n

worst => O(n)

Best => O(1)

```
// Maximum value in an array
public static int maxValue2(int[] arr) {
    int max = Integer.MIN_VALUE;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

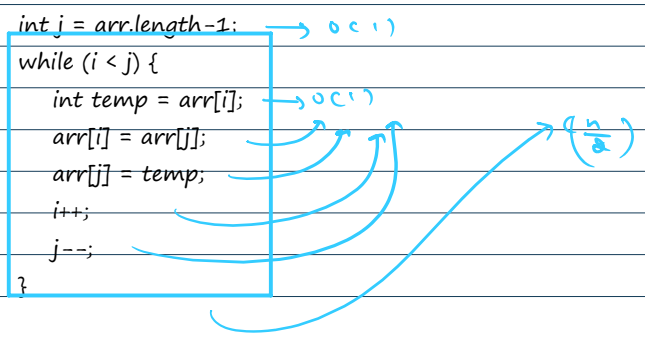
Best | worst => O(n)

```
// Reverse printing an array
public static void reversePrint(int[] arr) {
    for (int i = arr.length - 1; i >= 0; i--) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}
```

Best | worst => O(n)

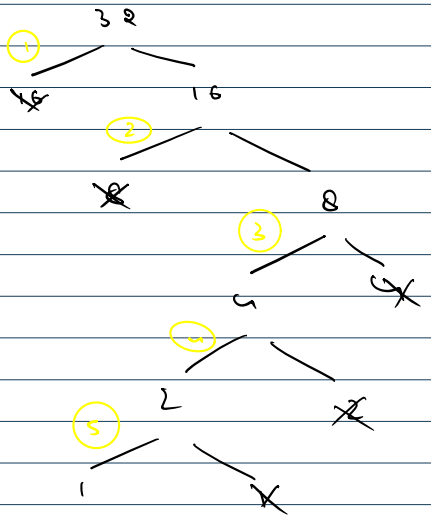
```
// Reversing an array
public static void reverseArray(int[] arr) {
    int i = 0;
```

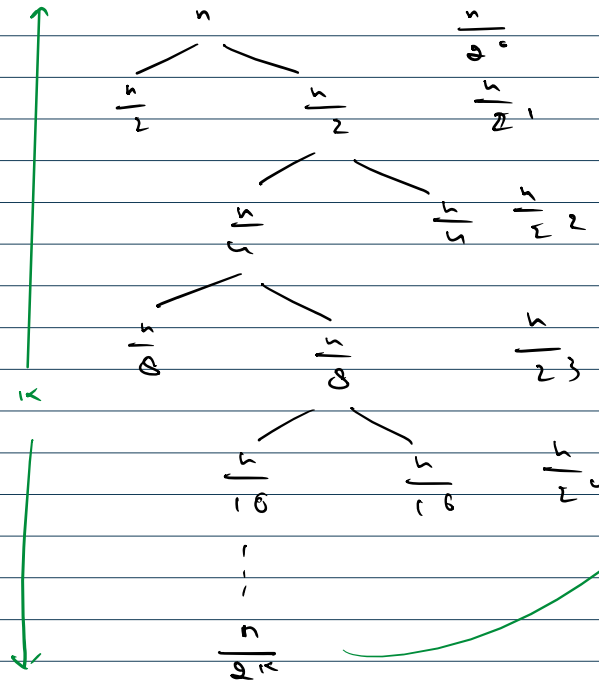
```
// Reversing an array
public static void reverseArray(int[] arr) {
    int i = 0;                          → O(1)
    int j = arr.length-1;               → O(1)
    while (i < j) {
        int temp = arr[i];              → O(1)
        arr[i] = arr[j];
        arr[j] = temp;
        i++;
        j--;
    }
}
```



Best / worst => O(n/2) / O(n)

```
// Binary Search
public static int binarySearch(int[] arr, int item) {
    int n = arr.length;
    int low = 0;
    int high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == item) {
            return mid;
        } else if (arr[mid] > item) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return -1;
}
```





$\log a^a = 1$

$\log b^k = k \cdot \log b$

$\frac{n}{2^k} = 1$

$n = 2^k$

$\log_2 n = \log_2 2^k$

$\log_2 n = k \cdot \log_2 2$

$\log_2 n = k$

$k = \log_2 n$

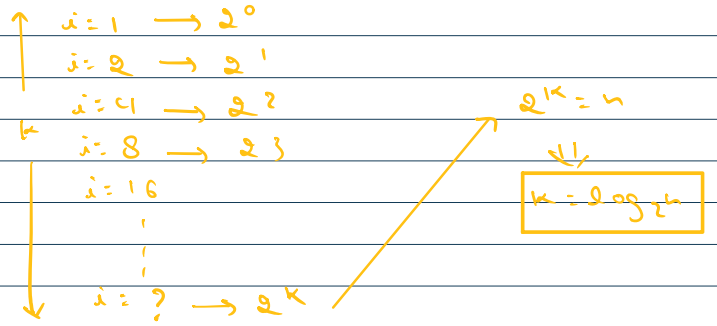
```
int n = 566789;     → O(1)
int i = 0;          → O(1)
```

Best / worst => O(n)

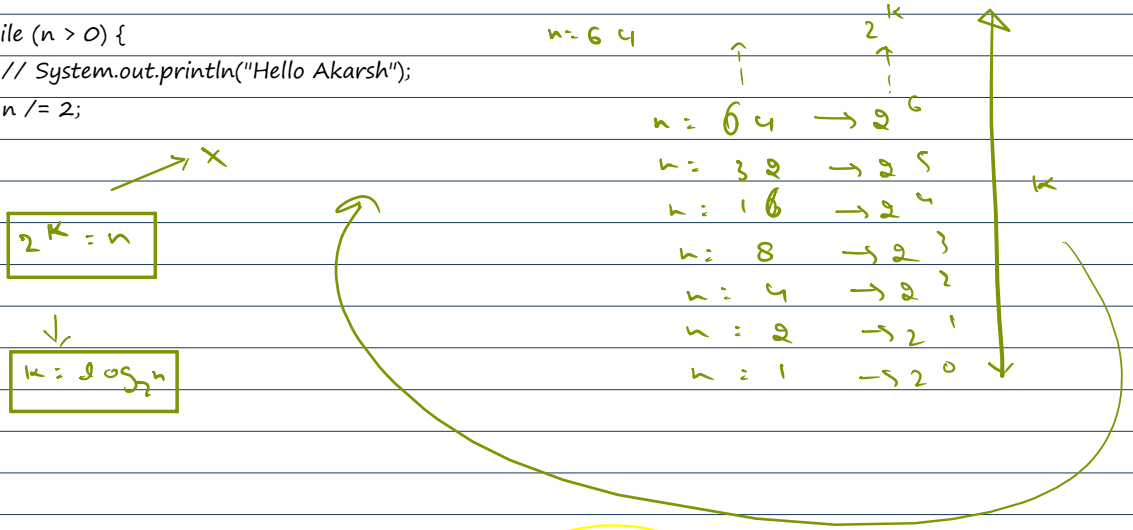
```
while (i < n) {
    // System.out.println("Hello Akarsh");
    i++;
}
```

→ O(n)

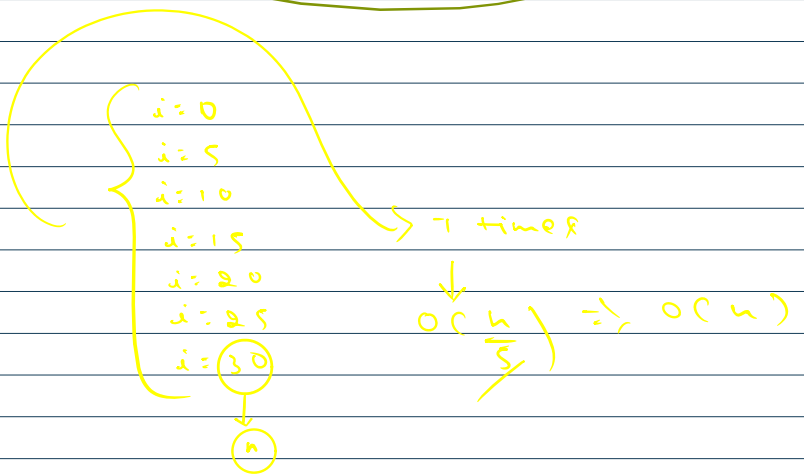
```
while (i <= n) {
    // System.out.println("Hello Akarsh");
    i *= 2;
}
```




```
while (n > 0) {
    // System.out.println("Hello Akarsh");
    n /= 2;
}
```

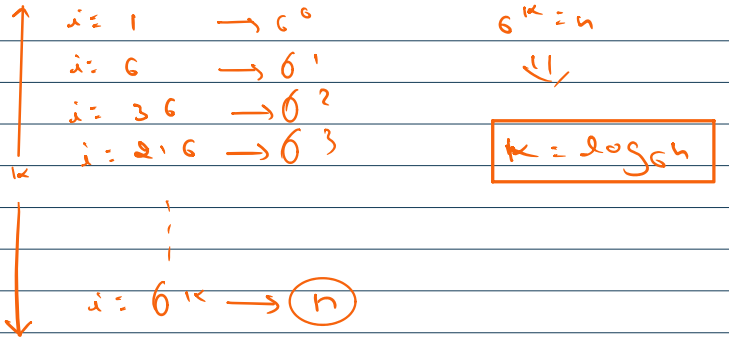



```
while (i <= n) {
    // System.out.println("Hello Akarsh");
    i += 2;
    i += 3;
}
```




```
while (i <= n) {
    // System.out.println("Hello Akarsh");
    i *= 2;
    i *= 3;
}
```

→ i *= 6



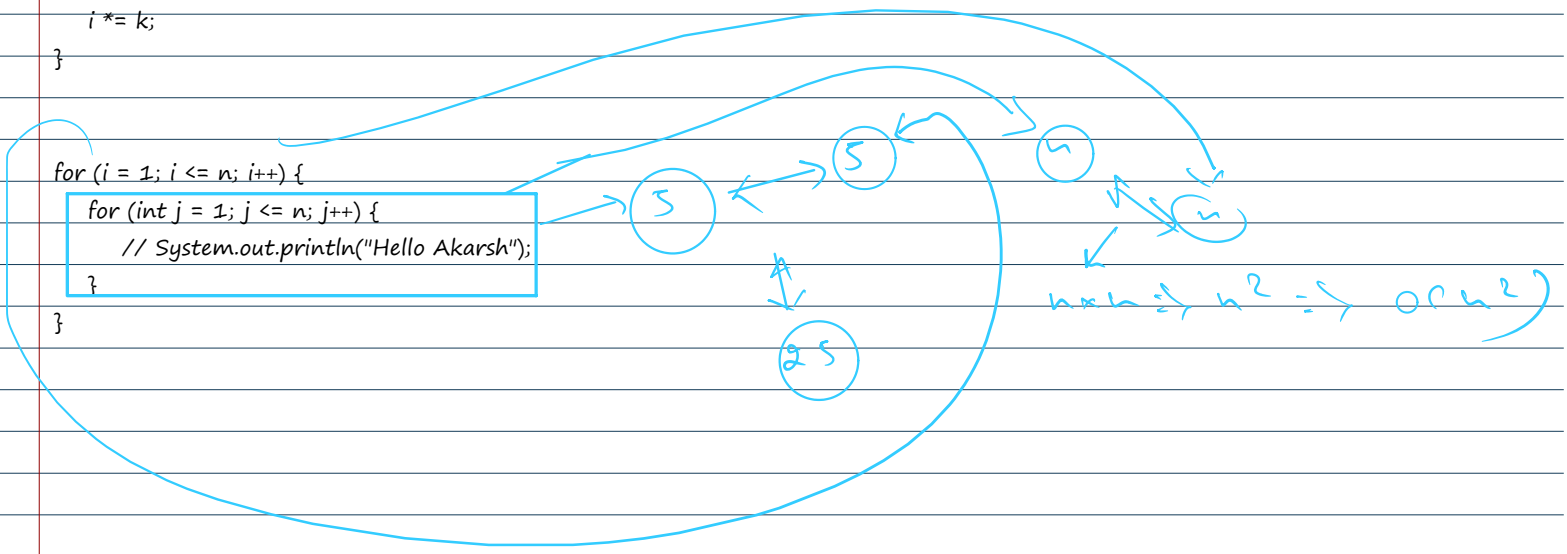
```
while (n > 0) {
    // System.out.println("Hello Akarsh");
    n /= 2;
    n /= 3;
}
```

→ n/=6 → $\log_6 n$

```
int k = 2;
while (i <= n) {
    // System.out.println("Hello Akarsh");
    i *= k;
}
```

→ $O(\log_k n)$

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // System.out.println("Hello Akarsh");
    }
}
```



Nested loops
- dependent
- independent

loop inside loop

```
for (i = 1; i * i <= n; i++) {
    // System.out.println("Hello Akarsh");
}
```

i * i = n
i <= √n
Time => O(√n)

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= i * i; j++) {
        for (k = 1; k <= n / 2; k++) {
            // System.out.println("Hello Akarsh");
        }
    }
}
```

i = 1 : $1^2 * \frac{n}{2}$ + i = 2 : $2^2 \times \frac{n}{2}$ + i = 3 : $3^2 \times \frac{n}{2}$ + i = 4 : $4^2 \times \frac{n}{2}$ + ... + i = n : $n^2 \times \frac{n}{2}$

$\frac{n}{2}[1^2 + 2^2 + 3^2 + \dots + n^2]$

$\frac{n}{2}\left[\frac{n(n+1)(2n+1)}{6}\right]$

=> $O(n^4)$

→ n/2

```
for (i = n / 2; i <= n; i++) {
    for (int j = 1; j <= n / 2; j++) {
        for (k = 1; k <= n; k = k * 2) {
            // System.out.println("Hello Akarsh");
        }
    }
}
```

n/2 → n/2 → log k n → (n/2) × (n/2) × log k n ⇒ $n^2 \log_k n$

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j += i) {
        // System.out.println("Hello Akarsh");
    }
}
```

i: 1 i: 2 i: 3 i: 4 - - - i: n

n n/2 n/3 n/4 n/n

$n\left[1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n}\right]$

$n \cdot \log n$

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j += i) {
        // System.out.println("Hello Akarsh");
    }
}
```

i: 1 i: 2 i: 3 i: 4 - - i: n

1/1 2/2 3/3 4/4 n/n

1 1 1 1 - - - 1

O(n) → add 1 n times

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j += 1) {
        // System.out.println("Hello Akarsh");
    }
}
```

i: 1 i: 2 i: 3 - - - i: n

1 + 2 + 3 + ... + n

→ sum of first n terms → add all these

$\frac{n(n+1)}{2}$ ⇒ $O(n^2)$

Space complexity

Extra space we use in program.

Reverse array using extra array $\Rightarrow O(n)$

Time $\Rightarrow$ 1 s

$\rightarrow 10^8$ instruction can execute

$n = 10^3 \Rightarrow O(n^3) \Rightarrow (10^3)^3 = 10^3 \cdot 10^3 \cdot 10^3 = 10^9$ ✗

$\rightarrow O(n^2) \Rightarrow (10^3)^2 = 10^6$ ✓

$O(n \log n)$ ✓

$n = 10^5 \Rightarrow$ ~~?~~ $O(n^2) \Rightarrow (10^5)^2 = 10^{10}$ ✗

$O(n \log_2 n) \Rightarrow 10^5 \cdot \log_2 10^5 = 10^5 \times 5 \cdot \log_2 10$ (3.32)

$= 10^5 \cdot 17$

$= 10^6 \cdot \times 1.7$ ✓